

Java File Handling & Streams

Simple English Notes — Definitions, Theory, Examples, Differences and Exercises

Topics covered: java.lang, File, FileReader, FileWriter, BufferedReader, BufferedWriter, FileInputStream, FileOutputStream, ObjectInputStream, ObjectOutputStream, Serialization, Deserialization, Serializable, exceptions, and assignment-style examples.

1. What is File Handling?

Definition: File handling means creating, reading, writing, updating, and managing files using a Java program.

Files are useful because data stored only in memory is normally lost when the program ends. A file provides persistent storage.

```
Java Program
↓
File Handling
↓
File
↓
Data remains after the program ends
```

2. java.lang Package

Definition: java.lang is a core Java package that contains fundamental classes used by almost every Java program. It is automatically imported.

You normally do not need to write import java.lang.*.

Important classes: String, System, Object, Math, Integer, Double, Exception, StringBuilder, Thread.

```
public class Main {
    public static void main(String[] args) {
        String name = "Ujjwal";
        System.out.println(name);
        System.out.println(Math.sqrt(25));
    }
}
```

Important: File handling classes such as FileReader, FileWriter, BufferedReader, BufferedWriter, FileInputStream and FileOutputStream belong to java.io, not java.lang.

3. Character Streams vs Byte Streams

Character streams are mainly used for text/character data. Examples: FileReader and FileWriter.

Byte streams read and write raw bytes. They are suitable for binary data such as images, PDFs, ZIP files and other non-text data. Examples: FileInputStream and FileOutputStream.

```
Text / Characters
File → FileReader → Program
Program → FileWriter → File

Bytes
File → FileInputStream → Program
Program → FileOutputStream → File
```

4. File Class

Definition: java.io.File represents a file or directory path. It can be used to check existence, get information, create/delete files, and work with paths.

```
import java.io.File;

File file = new File("employees.txt");

System.out.println(file.exists());
System.out.println(file.getName());
System.out.println(file.getAbsolutePath());
System.out.println(file.length());
```

Note: Creating a File object does not necessarily create the physical file. It represents the path.

5. FileReader

Definition: FileReader is a character stream used to read text from a file.

Use it when: you need to read text/characters from a file.

```
import java.io.FileReader;
import java.io.IOException;

FileReader fr = new FileReader("data.txt");

int data;
while ((data = fr.read()) != -1) {
    System.out.print((char) data);
}

fr.close();
```

The read() method returns an int. It returns -1 when the end of the file is reached.

6. FileWriter

Definition: FileWriter is a character stream used to write text/characters to a file.

```
import java.io.FileWriter;

FileWriter fw = new FileWriter("data.txt");

fw.write("Hello Java");
fw.write("\n");
fw.write("File handling");

fw.close();
```

Append mode: new FileWriter("data.txt", true) writes at the end instead of normally replacing existing content.

7. BufferedReader

Definition: BufferedReader is a buffered character input stream used to read text efficiently. Its most useful method is readLine(), which reads one complete line.

Why use it? It reduces direct I/O operations by using a buffer and makes line-by-line reading convenient.

```
import java.io.*;

BufferedReader br =
    new BufferedReader(
        new FileReader("employees.txt")
    );

String line;

while ((line = br.readLine()) != null) {
    System.out.println(line);
}
```

```
br.close();
```

If there are no more lines, `readLine()` returns null.

8. BufferedWriter

Definition: `BufferedWriter` is a buffered character output stream used to write text efficiently.

Useful methods: `write()`, `newLine()`, `flush()`, `close()`.

```
import java.io.*;

BufferedWriter bw =
    new BufferedWriter(
        new FileWriter("employees.txt")
    );

bw.write("Ujjwal,Manager,50000");
bw.newLine();
bw.write("Rahul,Engineer,40000");
bw.newLine();

bw.close();
```

`newLine()` moves to the next line. `close()` also flushes remaining buffered data and releases the resource.

9. FileInputStream

Definition: `FileInputStream` is a byte stream used to read raw bytes from a file.

Use it for: binary files, images, PDFs, ZIP files, or whenever raw byte access is required.

```
import java.io.*;

FileInputStream fis =
    new FileInputStream("data.bin");

int data;

while ((data = fis.read()) != -1) {
    System.out.print(data + " ");
}

fis.close();
```

10. FileOutputStream

Definition: `FileOutputStream` is a byte stream used to write raw bytes to a file.

```
import java.io.*;

FileOutputStream fos =
    new FileOutputStream("data.bin");

fos.write(65);
fos.write(66);
fos.write(67);

fos.close();
```

For text converted to bytes, you can use `text.getBytes()`, but for normal text files `FileWriter/BufferedWriter` are usually more convenient.

11. Copying a Binary File

FileInputStream and FileOutputStream can be combined to copy a binary file.

```
FileInputStream fis =
new FileInputStream("photo.jpg");

FileOutputStream fos =
new FileOutputStream("photo-copy.jpg");

int data;

while ((data = fis.read()) != -1) {
    fos.write(data);
}

fis.close();
fos.close();
```

In real applications, reading/writing a byte array buffer is generally more efficient than processing one byte at a time.

12. Serialization

Definition: Serialization is the process of converting a Java object's state into a serialized byte representation so it can be stored or transmitted.

```
Java Object
↓
Serialization
↓
Serialized bytes
↓
File / Stream
```

Serialization is useful when you want to save the state of a Java object and later reconstruct it.

13. Serializable Interface

A class normally implements `java.io.Serializable` to indicate that its objects can be serialized.

Serializable is a **marker interface**: it does not require you to implement methods.

```
import java.io.Serializable;

class Employee implements Serializable {
    String name;
    String role;
    double salary;

    Employee(String name, String role, double salary) {
        this.name = name;
        this.role = role;
        this.salary = salary;
    }
}
```

14. ObjectOutputStream

Definition: ObjectOutputStream writes Java objects to an output stream using serialization.

It is commonly placed on top of FileOutputStream.

```
Employee e =
new Employee("Ujjwal", "Manager", 50000);

ObjectOutputStream oos =
new ObjectOutputStream(
new FileOutputStream("employee.dat"))
```

```

);

oos.writeObject(e);
oos.close();

Java Object
↓
ObjectOutputStream
↓
FileOutputStream
↓
employee.dat

```

15. ObjectInputStream

Definition: ObjectInputStream reads serialized Java objects from an input stream and reconstructs them. This is called deserialization.

```

ObjectInputStream ois =
new ObjectInputStream(
new FileInputStream("employee.dat")
);

Employee e =
(Employee) ois.readObject();

ois.close();

System.out.println(e.name);
System.out.println(e.role);
System.out.println(e.salary);

```

readObject() returns Object, so a cast is commonly used when assigning it to a specific class type.

16. Serialization + Deserialization Complete Example

```

import java.io.*;

class Employee implements Serializable {
String name;
String role;
double salary;

Employee(String name, String role, double salary) {
this.name = name;
this.role = role;
this.salary = salary;
}
}

public class Main {
public static void main(String[] args) {

Employee employee =
new Employee("Ujjwal", "Manager", 50000);

// Serialization
try {
ObjectOutputStream oos =
new ObjectOutputStream(
new FileOutputStream("employee.dat")
);

oos.writeObject(employee);
oos.close();

```

```

} catch (IOException e) {
    System.out.println(e.getMessage());
}

// Deserialization
try {
    ObjectInputStream ois =
        new ObjectInputStream(
            new FileInputStream("employee.dat")
        );

    Employee e =
        (Employee) ois.readObject();

    ois.close();

    System.out.println("Name: " + e.name);
    System.out.println("Role: " + e.role);
    System.out.println("Salary: " + e.salary);

} catch (IOException | ClassNotFoundException e) {
    System.out.println(e.getMessage());
}
}
}

```

17. FileReader vs FileInputStream

FileReader	FileInputStream
Character stream	Byte stream
Designed for text/characters	Designed for raw bytes
Convenient for text files	Suitable for binary data
Examples: .txt, CSV text	Examples: images, PDFs, ZIPs, binary data
Reads characters	Reads bytes

18. FileWriter vs FileOutputStream

FileWriter	FileOutputStream
Character stream	Byte stream
Writes text/characters	Writes raw bytes
Convenient for text files	Suitable for binary data
Example: reports.txt	Example: image/data binary

19. Reader/Writer vs Object Streams

Reader/Writer	ObjectInput/OutputStream
Mainly text/character data	Serialized Java objects
Human-readable text can be stored	Serialized file is generally binary/not human-readable
FileReader/FileWriter	ObjectInputStream/ObjectOutputStream
Does not reconstruct arbitrary Java objects automatically	Can save and reconstruct Serializable object state
Good for text files and reports	Good when object state itself must be persisted

20. All Important Classes at a Glance

Class	Direction	Data	Main purpose
FileReader	File → Program	Characters	Read text
FileWriter	Program → File	Characters	Write text
BufferedReader	File → Program	Characters	Efficient text/line reading
BufferedWriter	Program → File	Characters	Efficient text writing
FileInputStream	File → Program	Bytes	Read raw/binary data
FileOutputStream	Program → File	Bytes	Write raw/binary data
ObjectInputStream	File/Stream → Program	Serialized objects	Deserialize objects
ObjectOutputStream	Program → File/Stream	Serialized objects	Serialize objects

21. Exception Handling in File Operations

File operations can fail because a file may not exist, permissions may be insufficient, the path may be wrong, or an I/O error may occur.

Common exceptions include `IOException`, `FileNotFoundException` and `ClassNotFoundException` for `readObject()`.

```
try {
    BufferedReader br =
        new BufferedReader(
            new FileReader("employees.txt")
        );

    String line;
    while ((line = br.readLine()) != null) {
        System.out.println(line);
    }

    br.close();

} catch (IOException e) {
    System.out.println("File error: " + e.getMessage());
}
```

22. Try-with-Resources

Try-with-resources automatically closes resources that implement `AutoCloseable`. It is preferred for many file operations.

```
try (
    BufferedReader br =
        new BufferedReader(
            new FileReader("employees.txt")
        )
) {
    String line;

    while ((line = br.readLine()) != null) {
        System.out.println(line);
    }

} catch (IOException e) {
    System.out.println(e.getMessage());
}
```

You do not need to explicitly call `br.close()` in this example.

23. Assignment Example — Employee File Handling

Suppose an assignment has `Manager`, `Engineer` and `SalesPerson` classes and asks you to save and load employee information.

```

interface Employee {
    void display();
}

class Manager implements Employee {
    private String name;
    private double salary;

    Manager(String name, double salary) {
        this.name = name;
        this.salary = salary;
    }

    public void display() {
        System.out.println(
            "Manager: " + name +
            " | Salary: " + salary
        );
    }
}

```

A text-file approach stores data in a readable format such as:

```

Manager,Ujjwal,50000
Engineer,Rahul,40000
SalesPerson,Amit,35000

```

Use `BufferedWriter` to save these lines and `BufferedReader` to read them later. After reading a line, split it using comma and create the appropriate object.

```

String[] data = line.split(",");

String type = data[0];
String name = data[1];
double salary = Double.parseDouble(data[2]);

Employee employee;

if (type.equals("Manager")) {
    employee = new Manager(name, salary);
} else if (type.equals("Engineer")) {
    employee = new Engineer(name, salary);
} else {
    employee = new SalesPerson(name, salary);
}

employee.display();

```

24. Exercises

Exercise 1 — `FileInputStream`: Create `data.txt` containing HELLO. Read it byte-by-byte using `FileInputStream` and print the characters.

Exercise 2 — `FileOutputStream`: Create `output.txt` and write Hello Java using `FileOutputStream`.

Exercise 3 — `FileReader/FileWriter`: Copy the contents of `input.txt` to `output.txt` using `FileReader` and `FileWriter`.

Exercise 4 — `BufferedReader`: Read `employees.txt` line-by-line and count the number of employees.

Exercise 5 — `BufferedWriter`: Write three employee records to `employees.txt`, one record per line.

Exercise 6 — `Serialization`: Create an `Employee` class implementing `Serializable`. Save one `Employee` using `ObjectOutputStream` and read it back using `ObjectInputStream`.

Exercise 7 — `Assignment level`: Create `Manager`, `Engineer` and `SalesPerson` implementing `Employee`. Save their text records using `BufferedWriter`, load them using `BufferedReader`, recreate objects, and call `display()`.

25. Quick Revision

FileReader → Read characters/text
FileWriter → Write characters/text

BufferedReader → Efficient text reading, especially line-by-line
BufferedWriter → Efficient text writing

FileInputStream → Read bytes
FileOutputStream → Write bytes

ObjectOutputStream → Serialize/save Java objects
ObjectInputStream → Deserialize/read Java objects

Serializable → Marker interface used to allow object serialization

Serialization → Object → serialized bytes
Deserialization → serialized bytes → Object

Golden rule: Text → Reader/Writer. Raw binary bytes → InputStream/OutputStream. Java object state → ObjectInputStream/ObjectOutputStream with Serializable.